Machine Learning Project

Road Accident Severity Prediction

Project Report: Check Your Understanding Answers and Conclusion

Submitted by	Swati Chaurasia
Project	Multi-class classification of road accident severity (Slight / Serious / Fatal Injury)
Dataset	Addis Ababa Sub-city Police Department road traffic accidents, 2017-2020 (12,316 records, 32 columns)
Target column	Accident_severity
Models compared	Logistic Regression, Decision Tree, Random Forest (all with class_weight='balanced')

Step 1: Set Up Your Python Environment

Q1. What is the purpose of importing a library? What happens if you use its function without importing?

Importing loads a library's ready-made code into the notebook so I can call its functions instead of writing everything myself. If I use a library function without importing it, Python raises a *NameError* (for example, "name 'pd' is not defined"). If the library is not installed at all, the import itself fails with *ModuleNotFoundError*.

Q2. Difference between pandas and numpy?

numpy works with fast arrays of numbers and is used for mathematical operations. pandas is built on top of numpy and adds labelled tables (DataFrames) with named columns, mixed data types (text and numbers), CSV reading, missing-value handling and filtering. In short: numpy for number crunching, pandas for working with real-world tabular data.

Q3. Which library draws a bar chart or heatmap?

seaborn (for example countplot, barplot and heatmap), which is built on matplotlib. matplotlib is used for figure size, titles and showing the plot.

Q4. Two things scikit-learn provides that this project needs.

(1) Models: LogisticRegression, DecisionTreeClassifier and RandomForestClassifier.

(2) Tools: train_test_split for splitting data, LabelEncoder for encoding, and evaluation metrics such as accuracy_score, confusion_matrix and classification_report.

Step 2: Load the Dataset

Q1. What is a CSV file? Why is it common?

CSV (Comma Separated Values) is a plain text file where each line is one record and the values in it are separated by commas; the first line holds the column names. It is common because it is simple, small, human-readable and opens in almost every tool (Excel, Python, R, databases) without special software.

Q2. What is a pandas DataFrame? Describe it to an Excel user.

A DataFrame is like an Excel sheet inside Python: rows and named columns. The difference is that I control it with code, so I can clean, filter and transform thousands of rows in one line, and every column has a data type.

Q3. Why preview the first rows right after loading?

It confirms the file loaded properly: correct headers, correct separators, sensible values and the expected columns. Catching a wrong or corrupted load now saves me from building a model on bad data.

Q4. If the dataset link was broken, what would I see? pd.read_csv would raise an error such as *HTTPError: HTTP Error 404: Not Found* (or a URLError if there is no connection), and no DataFrame would be created.

Step 3: Explore the Dataset (EDA)

Q1. Why explore before building a model?

EDA shows the size, data types, missing values and class balance. These facts decide how I clean the data, which metrics I use and which problems (like imbalance) I must handle. Skipping it means feeding unknown problems into the model (garbage in, garbage out).

Q2. Slight vs Fatal accidents: what did I notice, what is it called, why a problem?

Slight Injury dominates (about 10,400 of 12,316 records, roughly 85%), Serious Injury is about 1,700 (14%), and Fatal Injury is only about 158 (roughly 1%). This is **class imbalance**. It is a problem because a model can score high accuracy by mostly predicting the majority class while failing on the rare but most important Fatal cases.

Q3. Two columns with the most missing values; why might police records miss them?

Usually Defect_of_vehicle and Service_year_of_vehicle (followed by Work_of_casuality and Fitness_of_casuality). Officers at an accident scene often cannot check a vehicle's defects or age, the vehicle owner may be unknown (hit-and-run), forms are filled in under time pressure, and some fields are simply left blank.

Q4. Categorical vs numerical column, with one example each.

A categorical column holds labels or groups (example: Weather_conditions with values like Normal, Raining). A numerical column holds measured quantities you can do arithmetic on (example: Number_of_vehicles_involved or Number_of_casualties).

Q5. What did the bar chart show that a list of numbers did not?

It shows the imbalance instantly: one tall bar for Slight Injury and an almost invisible bar for Fatal Injury. The size of the gap is felt immediately, whereas in a list I have to read and compare the numbers.

Step 4: Clean the Dataset (Handle Missing Values)

Q1. Why can't an ML algorithm work directly with NaN values?

Models compute with numbers (weights, distances, split thresholds). NaN is "not a number", so such calculations fail or spread NaN through the results. Most scikit-learn models raise a ValueError when they see NaN.

Q2. Why fill missing values instead of deleting rows?

Several columns are missing thousands of values, so dropping every row with any gap would delete a large share of the 12,316 records, including the already tiny number of Fatal cases. Filling keeps the rows and their other useful information.

Q3. What is a median? Why safer than the mean?

The median is the middle value after sorting. The mean is pulled towards extreme values (one huge or tiny number shifts it), while the median stays close to a typical value, so it is a safer fill for numeric columns with outliers.

Q4. One disadvantage or risk of imputation.

Imputed values are guesses, not real observations. They can hide real patterns, add bias, and make the model look more certain than it should be. Also, "Unknown" lumps together many different real situations into one category.

Step 5: Feature Engineering and Feature Selection

Q1. What is feature engineering? Why convert Time to Hour?

Feature engineering means creating or transforming columns so that useful patterns become easier for the model to see. An exact time like 17:02:00 is almost unique for each row, so the model cannot generalise from it. The Hour (0 to 23) groups accidents into meaningful slots such as morning rush, evening traffic or night-time low visibility.

Q2. Why exclude after-the-fact casualty columns?

Columns like Casualty_class, Sex_of_casualty and Fitness_of_casuality describe the victim, who is only known after the accident has happened. They would not be available when we want to predict severity.

Q3. What could go wrong if we included information known only after severity is determined (data leakage)?

Data leakage means information about the answer slips into the inputs. The model would score unrealistically well in testing, but in real use those inputs would not exist yet, so it would fail. The results would be misleadingly good and the model useless in practice. (Be ready to justify borderline columns such as Number_of_casualties, which is also only known after the accident.)

Q4. One new feature from combining two columns.

An *Is_weekend* or *Rush_hour* flag from Day_of_week and Hour, or a Day_of_week x Hour combination. It could help the model learn patterns such as Friday or weekend late-night accidents being more severe because of speed, fatigue and low visibility.

Step 6: Encode Categorical Columns

Q1. Why can't algorithms understand text categories?

Algorithms do arithmetic: multiplying by weights, measuring distances, comparing against thresholds. None of that can be done on words like "Raining", so categories must become numbers first.

Q2. How does Label Encoding work?

LabelEncoder looks at the unique values in a column, sorts them (alphabetically) and assigns 0, 1, 2... to them. fit_transform() learns the mapping and replaces the text; inverse_transform() converts numbers back to the original labels.

Q3. Why encode the target separately?

The target has its own encoder so I can keep its mapping and later convert predictions back to "Slight / Serious / Fatal Injury". Feature encoders are kept in their own dictionary, so nothing gets mixed up.

Q4. Can Label Encoding mislead a model? What is One-Hot Encoding?

Yes. The numbers 0, 1, 2 imply an order (2 > 1 > 0) even for categories with no natural order, like weather types. This can mislead models such as Logistic Regression (tree-based models are much less affected). One-Hot Encoding instead creates a separate 0/1 column for each category, so no order is implied, at the cost of more columns.

0 = Fatal injury, 1 = Serious Injury, 2 = Slight Injury).

Step 7 and 8: Prepare X / y and Split into Train and Test

Q1. Difference between X and y?

X holds the input features (the "question"); y holds the target to predict, Accident_severity (the "correct answer").

Q2. Why split into training and testing sets?

To check how the model performs on data it has never seen. The training set (about 80%) teaches the model; the testing set (about 20%) gives an honest estimate of real-world performance.

Q3. What does stratify do, and why is it important here?

stratify=y keeps the same class proportions in both train and test sets. With only about 1% Fatal cases, a plain random split could leave too few (or even no) Fatal examples in one of the sets, making training or evaluation unreliable.

Q4. Problem if we test on the training data?

The score is falsely high because the model has already seen those answers (it may simply memorise them, especially a Decision Tree). It would hide overfitting and tell me nothing about new accidents.

Q5. Why fix random_state?

It makes the shuffling repeatable, so I (or an evaluator) get the same split and the same results every time, which also makes comparing models fair.

Step 9: Train Machine Learning Models

Q1. How does a Decision Tree make a prediction?

It asks a chain of yes/no questions about the features (for example "Is Light_conditions dark?", "Are more than 2 vehicles involved?"), moving down branches until it reaches a leaf. The leaf gives the class that was most common among training rows that ended there.

Q2. Why does a Random Forest usually beat a single tree?

A forest trains many trees on random subsets of rows and features and combines them by voting. Individual trees make different mistakes, which cancel out, so the forest overfits less and is more stable.

Q3. What does class_weight='balanced' do, and why here?

It gives each class a weight inversely proportional to how common it is, so mistakes on rare classes (Fatal, Serious) cost more during training. Because Fatal is only about 1% of the data, this stops the model from ignoring it.

Q4. What does the model learn from during fitting?

It learns the relationship between the features in X_train and the answers in y_train: Logistic Regression learns coefficients (weights), a Decision Tree learns split rules, and a Random Forest learns many such trees. The goal is general patterns, not memorising rows.

Step 10: Evaluate the Models

Model comparison (fill in from your notebook)

Model	Accuracy	Recall (Fatal)	Macro F1
Logistic Regression	[FILL IN]	[FILL IN]	[FILL IN]
Decision Tree	[FILL IN]	[FILL IN]	[FILL IN]
Random Forest	[FILL IN]	[FILL IN]	[FILL IN]

Q1. Why can accuracy alone mislead on this dataset?

About 85% of records are Slight Injury, so a model that always predicts Slight Injury would be about 85% accurate while catching no Fatal or Serious accidents. Accuracy hides failure on the rare classes.

Q2. Define Precision and Recall. Which matters more for Fatal accidents?

Precision: of everything predicted as Fatal, how many really were Fatal. Recall: of all real Fatal accidents, how many the model caught. For Fatal accidents recall matters more, because missing a real fatal case (a false negative) means a slower emergency response, while a false alarm only wastes some resources. Precision still matters so alarms stay useful, and F1 balances the two.

Q3. What do the diagonal and off-diagonal values of a confusion matrix mean?

The diagonal shows correct predictions for each class (actual = predicted). Values outside it are mistakes: rows are actual classes, columns are predicted classes, so each off-diagonal cell shows which class was confused with which.

Q4. Which model performed best overall and why?

Random Forest usually performs best on tabular data like this because combining many trees reduces overfitting and captures non-linear feature interactions that Logistic Regression cannot, while a single Decision Tree overfits.

Q5. Would I trust a model with 95% accuracy on this dataset?

Not immediately. I would check the majority-class baseline (about 85%), per-class precision/recall/F1 and the confusion matrix, whether data leakage exists, whether the score is on unseen test data, and how well it catches Fatal cases.

Step 11: Feature Importance

Q1. What does feature importance tell us?

It shows how much each input feature contributed to the Random Forest's decisions (how much it helped the trees split the data well), as relative scores that sum to 1. A higher score means the model relied on that feature more.

Q2. Which features were most important? Does it match intuition?

Real-life intuition says factors like time of day / light conditions, cause of accident, type of collision, number of vehicles, road surface and driver experience should matter. Compare this with my chart and comment on agreement or surprises.

Q3. How could traffic police or city planners use this?

Police can focus patrols and enforcement on high-risk hours and the top causes of severe accidents. Planners can fix risky junction types, lighting and road surfaces, and authorities can design targeted safety campaigns, all based on data rather than guesswork.

Q4. Can Logistic Regression give similar insight?

Yes, through its coefficients: each feature gets a weight per class, and its sign and size show the direction and strength of influence. They are harder to interpret here because label-encoded categories are not true numeric scales and features are not scaled.

Step 12: Prediction on New (Sample) Data

Q1. Why convert the numeric prediction back to a text label?

A number like 2 means nothing to a police officer or planner. inverse_transform on the target encoder turns it back into "Slight Injury", "Serious Injury" or "Fatal Injury".

Q2. Was my sample prediction correct? If wrong, why?

If wrong, likely reasons: the rare classes have few examples; the available columns miss key details such as speed or impact force; missing values were filled with "Unknown"; and similar-looking accidents can have different outcomes.

Q3. Where would new accident data come from in real deployment?

From a digital accident reporting system used by traffic police, emergency call centre (112/108-type) reports, or traffic control rooms and field apps. The new record must go through the same cleaning and the same saved encoders (transform, not fit) before prediction.

Step 13: Conclusion

Summary of the project

Problem solved. I built a multi-class classification model that predicts road accident severity (Slight, Serious or Fatal Injury) from driver, vehicle, road and weather conditions. It matters because early knowledge of likely severity lets emergency services respond faster and helps planners find dangerous conditions and locations.

Steps followed.

(1) Imported pandas, numpy, matplotlib, seaborn and scikit-learn.

(2) Loaded the Addis Ababa RTA CSV into a DataFrame.

(3) Explored shape, data types, class distribution and missing values.
(4) Filled categorical gaps with "Unknown" and numeric gaps with the median.
(5) Created the Hour feature from Time and selected only pre-accident features to avoid leakage.
(6) Label-encoded features and the target separately.
(7-8) Built X and y and made a stratified 80/20 train-test split.
(9) Trained Logistic Regression, Decision Tree and Random Forest with class_weight='balanced'.
(10) Evaluated with accuracy, confusion matrix and classification report.
(11) Checked Random Forest feature importance.
(12) Predicted on one unseen test row and decoded the label.

Check Your Understanding: Step 13

Q1. Two limitations I noticed.

(1) Severe class imbalance: Fatal cases are only about 1% of the data, so recall for Fatal Injury is hard to keep high even with class weights. (2) Data quality: many columns had thousands of missing values filled with "Unknown" or the median, and important causes of severity (speed, vehicle damage, impact) are not recorded. Also, the data comes from one city and period (2017-2020), so it may not generalise elsewhere, and Label Encoding implies an order that does not really exist.

Q2. One realistic improvement for rare Fatal cases.

Apply SMOTE (synthetic oversampling) to the training set only, never the test set, and tune the model with GridSearchCV scoring on macro F1 or Fatal recall. Trying XGBoost and One-Hot Encoding for the categorical features would also likely help.

Q3. Real-world value in 3-4 sentences for a non-technical person.

This project teaches a computer to learn from thousands of past road accident reports so that it can estimate how serious a new accident is likely to be. With that early warning, ambulances and police can prioritise the most dangerous cases and arrive faster. The same model also points out which conditions, such as time of day or the cause of the accident, are linked to the worst outcomes, so authorities can target road safety improvements where they will save the most lives.